

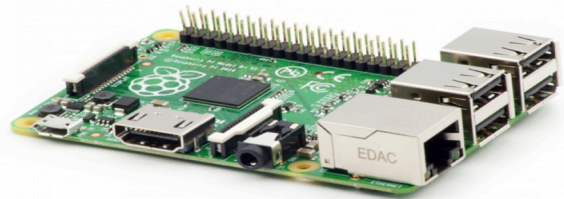
Raspberry Pi : un nano-ordinateur

- I. Présentation du Raspberry Pi
- II. Les bases du langage Python
- III. Les modules, la programmation objet : les classes et les méthodes
- IV. Les interfaces graphiques
- V. Interaction avec le monde physique extérieur avec GPIO

I. Présentation du Raspberry Pi 2

Il est équipé d'un circuit intégré unique regroupant :

- le CPU (Central Processing Unit) : un **microprocesseur** Broadcom BCM2836 ARM11 de 700 MHz
- le GPU (Graphics Processing Unit) : un **processeur graphique** et tous les circuits annexes nécessaires.



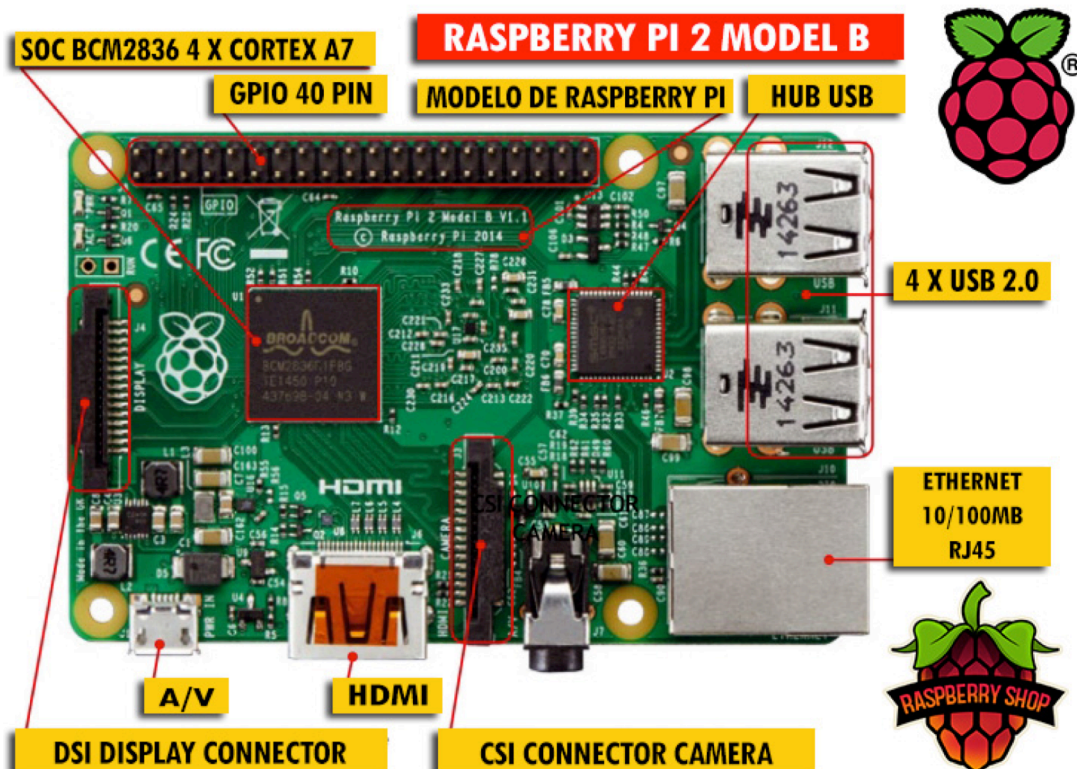
La mémoire RAM de 1 Go se trouve sous la carte.

Il possède des sorties USB sur lesquelles on peut brancher un clavier et une souris ainsi qu'une sortie vidéo HDMI (High-Definition Multimedia Interface) que l'on connectera à l'écran de l'ordinateur grâce à un adaptateur HDMI-VGA (il possède également un connecteur Ethernet RJ-45 LAN permettant la connexion à Internet).

Le Raspberry Pi utilise comme **disque dur** une carte Micro SD de 2GB qui contient le système d'exploitation ou OS (Operating System) open source **Linux**.

2 rangées de broches constituent ce qui est appelé le **GPIO** (General Purpose Input/Output) qui va nous permettre ultérieurement d'interagir avec le monde extérieur.

Description :



La distribution Linux utilisée est appelée Raspbian.

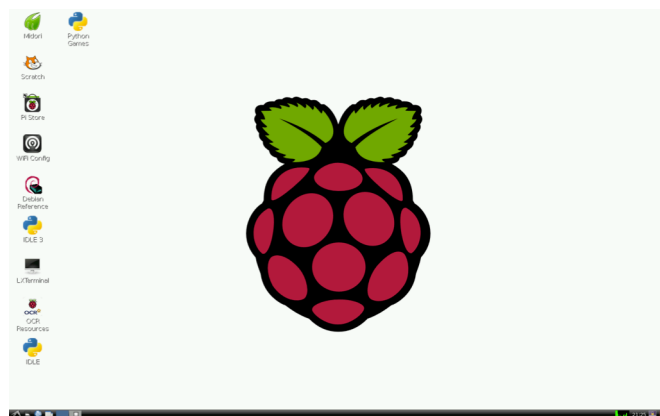
Il faut d'abord se connecter :

login : pi

password : raspberry

Puis écrire : startx pour atteindre la page d'accueil

Le bouton supérieur gauche Menu, comme le bouton inférieur gauche Start sous Windows, nous permet d'accéder à des applications préinstallées.



On trouve à droite du bouton Menu

- le Epiphany Web Browser : navigateur sous Linux
- le File Manager, équivalent de File Explorer sous Windows ou de Finder sous Mac ; il permet d'organiser les documents : les copier, les couper, les renommer, les mettre à la poubelle...
On observe que l'on se trouve actuellement à la racine /home/pi.
- LXTerminal : l'équivalent de command line sous Windows ou de Terminal sous Mac

Quelques commandes utiles :

- \$pwd (print working directory): indique le répertoire de travail actuel
- \$ls (list) : fait la liste des dossiers dans le répertoire de travail actuel
- \$cd (change directory) : permet d'accéder à un répertoire
- \$sudo (super-user do) : pour exécuter quelque chose
- \$touch : créer un fichier Python : touch myprogram.py
- \$nano : éditer un fichier Python : sudo nano myprogram.py
- \$rm : effacer un fichier Python : rm myprogram.py

II. Les bases du langage Python

Pour se lancer tout de suite dans le langage Python, on va écrire, comme il est de coutume quand on découvre un nouveau langage de programmation, notre premier programme «Hello World ».

On lance pour cela, à partir du menu, le programme appelé Python 3 ou Python Shell.

Ce programme permet de lancer des commandes en Python et de voir directement ce qu'elles font.

On lance des commandes à partir de >>> et la console de Python nous donne le résultat en dessous.

On peut commencer avec un peu d'arithmétique :

```
>>>2+2 puis Enter → 4
```

```
>>>5%3 puis Enter → 2
```

```
>>>2 ** 8 puis Enter → 256
```

```
>>>1<3 puis Enter → True
```

N.B : pour modifier le calcul d'une ligne, se placer sur celle-ci et faire Enter, la ligne est réécrite

Pour écrire des programmes, on crée un nouveau fichier à partir de File New Window pour utiliser le IDLE Editor :

Une ligne de commentaire est toujours très utile : ceci est notre premier programme en Python !

"""ceci est aussi un commentaire mais

sur plusieurs lignes

"""

```
print ('Hello')
print ('World')
```

Nous allons maintenant utiliser le File Manager pour créer un dossier appelé Python dans lequel on va enregistrer nos programmes écrits en Python : File Create New Folder

On enregistre notre programme appelé hello.py en revenant à l'éditeur : File Save As, choisir le Directory : /home/pi/Python

Pour faire fonctionner le programme, aller dans l'éditeur à Run Module ou F5, et le résultat s'affiche dans le Python Shell.

Si l'on ferme l'éditeur, tout est perdu mais on peut rappeler le programme dans le Python Shell en faisant File Open.

On peut maintenant voir quelques éléments de programmation en Python

1. Utilisation des variables

Assigner une valeur à une variable puis la réutiliser :

```
k=3/4
m=2+k
print (m)                Ctrl S (save) puis F5 (run)
```

2. Utilisation de for pour faire une boucle

```
for x in range(1,11,2):
    print(x)              → 1,3,5,7,9 : de 1 inclus à 11 exclu par pas de 2
```

Ici pas d'accolades comme dans JavaScript, mais la deuxième ligne est **indentée** (avec la touche Tab) par rapport à la première pour indiquer que l'instruction print doit se faire à l'intérieur de la boucle.

N.B : on observera que la fonction range inclue la valeur de départ mais exclue celle d'arrivée.

Création d'un nombre aléatoire

```
# on doit d'abord importer la librairie qui permet à Python de générer des nombres aléatoires
import random
x=random.randint(1,6)
print (x)
```

3. Les conditions avec if

On va faire ici un petit programme qui tire 5 fois de suite un nombre aléatoire entre 1 et 10 et affiche à chaque fois si le nombre sorti est pair ou impair.

```
import random
for i in range(1,6):
    x=random.randint(1,10)
    if x%2 == 0 :
        print ("nombre pair",x)
    else :
        print ("nombre impair",x)
```

4. Utilisation de while pour faire un autre type de boucle

On va faire ici un petit programme qui tire au sort un nombre compris entre 1 et 6 et s'arrête dès qu'il a tiré un 6.

```
import random
x=0
while True:
    x=random.randint(1,6)
    print (x)
    if x==6 :
        print('un six est sorti !')
        break
    # break permet de sortir d'une boucle
```

5. Les chaînes de caractères ou strings

- Connaître le nombre de caractères dans une chaîne

```
string="ma chaine"
print (len(string))      → 9
```

- Repérer les lettres dans un string

```
string="ma chaine"
print (string[0])        → m
'e' in string             → True
string.index('a')         → 1
string.count('a')         → 2   nombre d'occurrences de la lettre 'a'
```

- Extraire une partie de la chaîne

```
string="raspberry_Pi"
print (string[0:9])      → raspberry : de l'index 0 inclus à l'index 9 exclu
```

- Concaténation (addition des séquences)

```
#Entre deux strings
string1="Programmation avec "
string2="Raspberry Pi"
print (string1+string2)   → « Programmation avec Raspberry Pi »
string2*3                 → 'Raspberry PiRaspberry PiRaspberry Pi'
```

```
#Pour concaténer un string et une valeur numérique, il faut convertir la valeur numérique en string avec str()
string1="valeur : "
valeur =50
affichage=string1+str(valeur)
print(affichage)          → valeur : 50
```

6. Les listes ou arrays

```
liste=[23,48,5,78,90,2]
print (liste[0])          → 23
liste[0]=14
print (liste[0])          → 14 (une liste est un objet mutable contrairement à un string)
```

Des exemples de méthodes agissant sur les listes :

```

liste.sort()           → [2, 5, 14, 48, 78, 90] : ordonne la liste
print (liste)
liste.pop()            → [2, 5, 14, 48, 78] : retire le dernier élément de la liste
print (liste)
liste.pop(1)           → [2, 14, 48, 78] : retire l'élément d'index 1
print (liste)
liste.insert(1,5)
print (liste)          → [2, 5, 14, 48, 78] : insère 5 à l'index 1
print (len(liste))     → 5 : donne la longueur de la liste
print (liste[1:3])     → [5, 14] : garde les éléments de la liste de l'index 1 inclus à l'index 3 exclu

```

Savoir transformer une liste de chaînes en chaîne peut être utile :

```

liste=["Hello","how","are","you","?"]
print(' '.join(liste)) → séparation par 1 espace : Hello, how are you ?

```

7. Les fonctions

On crée ici une fonction qui ajoute la forme de politesse « s'il te plaît » à une demande. Ces deux procédures donnent le même résultat

```

def marque_de_politesse1(demande):
    demande_polie1=demande+" s'il te plait"
    print(demande_polie1)
marque_de_politesse1("Passe-moi le sel")

```

```

def marque_de_politesse2(demande):
    demande_polie2=demande+" s'il te plait"
    return(demande_polie2)
print(marque_de_politesse2("Passe-moi le sel"))

```

N.B : Il faut différencier les variables locales dont la portée est limitée à une fonction, des variables globales qui sont accessibles dans la totalité du programme, on le précisera en indiquant « global » devant la variable dans la fonction.

```

total=4
def ajout(n):
    global total
    total += n
    return total
print (ajout(6))      → 10

```

Première application utilisant les 7 premiers éléments de programmation : réaliser un jeu de pendu. Au départ, tirage au sort d'un mot dans une liste en comportant une dizaine. Le jeu propose le mot où toutes les lettres sont remplacées par un trait et séparées par un espace. L'utilisateur a droit à un maximum de 8 propositions pour retrouver le mot ; les bonnes lettres proposées remplacent progressivement les traits dans le mot à deviner. Indications :

- la création d'un array vide comportant par exemple 3 éléments : `mot_cache=[None]*3`
- un nombre `i` est paire si `i%2=0`, impaire si `i%2=1`
- déclarer une variable entrée par l'utilisateur : `proposition = input("Entrez une proposition : ")`

Résultat à atteindre :

```
- - - - -
Votre proposition ?a
- - - - - Encore 7 essais.
Votre proposition ?r
r - - - - - Encore 6 essais.
Votre proposition ?e
r e - - - e Encore 5 essais.
Votre proposition ?p
r e p - - e Encore 4 essais.
Votre proposition ?g
r e p - - e Encore 3 essais.
Votre proposition ?t
r e p t - e Encore 2 essais.
Votre proposition ?i
r e p t i - e Encore 1 essais.
Votre proposition ?l
| r e p t i l e Encore 0 essais.
Bravo !
```

8. Les dictionnaires

Les dictionnaires sont plus rapides et plus efficaces que les listes pour rechercher des données. Contrairement aux listes, les valeurs sont affectées à des clés et non à des index.

```
somme_a_partager={'Anne':10,'Paul':20,'Pierre':15}
print (somme_a_partager['Paul'])           → 20
somme_a_partager['Paul']=30
print (somme_a_partager)                   → {'Paul': 30, 'Pierre': 15, 'Anne': 10}
```

III. Les modules, la programmation objet : les classes et les méthodes

Un **module** est un programme que l'on peut appeler dans un autre programme. Nous avons déjà rencontré le module random

```
import random
x=random.randint(1,6)
```

Une autre manière de procéder

```
import random as r
x=r.randint(1,6)
```

Ou

```
#Pour tout importer du module
from random import *
x=randint(1,6)
```

Tous les modules inclus dans Python constituent la **librairie standard de Python** que l'on peut trouver ici : <https://docs.python.org/release/3.1.5/library/index.html>

Les modules les plus utiles sont :

- string : pour manipuler des chaînes de caractères
- datetime : pour manipuler des dates et des heures
- math : les fonctions mathématiques (sin, cos ...)
- pickle : pour conserver et restaurer des données
- urllib.request : pour lire des pages web
- tkinter : pour réaliser des interfaces graphiques

Installation de nouveaux modules : en plus de la librairie standard, des milliers de modules provenant de la communauté Python peuvent être installés. L'un d'entre eux très populaire est le module pygame.

Nous serons amenés ultérieurement à importer le module RPi.GPIO pour contrôler le GPIO (general purpose input/output) : *Import RPi.GPIO as GPIO* qui est déjà préinstallé.

Programmation Objet : classes et méthodes

Un exemple simple rencontré précédemment nous a déjà amené à utiliser une classe et une méthode préexistantes :

```
chaine="ma chaine"  
# on va indiquer la classe correspondant à chaine (double underscore)  
print(chaine.__class__) → <class 'str'>  
# on va compter le nombre de 'a' dans la chaine  
x=chaine.count('a')  
print(x) → 2
```

chaine est une **instance** de la **classe** string et count est une **méthode** appliquée à cette classe.

On peut créer nos propres classes.

Dans l'exemple ci-dessous, nous allons créer une classe appelée **Conversion** ; elle sera caractérisée par son facteur. Une première méthode associée à cette classe, appelée convertir, consistera à multiplier une valeur par ce facteur. Une autre méthode, appelée metaucarre, consistera à mettre au carré cette valeur.

On crée une instance c1 de cette classe définie par le facteur de conversion 5. On applique les méthode convertir et metaucarre à la valeur 2.

On définit ensuite une autre classe appelée **ConversionEtDecalage** qui héritera de la classe Conversion sa méthode metaucarre mais qui réécrit sa propre méthode convertir qui possède la propriété decalage.

```
class Conversion :  
    def __init__(self,facteur):  
        self.facteur=facteur  
  
    def convertir(self,value):  
        return 'Le resultat de la conversion de '+str(value)+' par le facteur '+str(self.facteur)+' est '+str(value*self.facteur)  
    def metaucarre(self,value):  
        return 'la mise au carre de '+ str(value)+' donne ' + str(value*value)  
  
c1=Conversion(5)  
  
print (str(c1.convertir(2)))  
print (str(c1.metaucarre(2)))
```

```

class ConversionEtDecalage(Conversion):
    def __init__(self,facteur,decalage):
        Conversion.__init__(self,facteur)
        self.decalage=decalage

    def convertir(self,value):
        return 'Le resultat de la conversion de '+str(value)+' par le facteur '+str(self.facteur)+' et le decalage '+str(self.decalage)+' est '+str(value*self.facteur+self.decalage)

c2=ConversionEtDecalage(5,3)
print (str(c2.convertir(2)))
print (str(c2.metaucarre(3)))

```

On obtient le résultat suivant :

Le resultat de la conversion de 2 par le facteur 5 est 10
 la mise au carre de 2 donne 4
 Le resultat de la conversion de 2 par le facteur 5 et le decalage 3 est 13
 la mise au carre de 3 donne 9

On peut maintenant appelé ce fichier conversion.py et appeler ce module dans un autre fichier placé dans le même dossier en écrivant `from conversion import *`

IV. Les interfaces graphiques

Tkinter est un outil permettant de réaliser une application avec une interface graphique (GUI : graphical user interface). Cette interface est préinstallée avec Python. Nous allons utiliser ici le widget Label

```

from tkinter import *

fenetre = Tk()

label = Label(fenetre, text="Hello World")
label.pack()

fenetre.mainloop()

```

On commence par assigner la variable fenetre à l'objet Tk.
 On crée une instance label de la classe Label dont l'argument est fenetre.
 La méthode pack est appelée sur label, cela dit au label de se placer sur l'espace disponible de la fenêtre.

Affichage de l'heure courante dans un widget Label

```

from tkinter import *

import time
def maj():
    # on arrive ici toutes les 1000 ms
    heure.set(time.strftime('%H:%M:%S'))
    fenetre.after(1000,maj)

fenetre = Tk()

```

```

fenetre.title("Heure courante")

# Creation d'un widget Label
heure = StringVar()
label=Label(fenetre,textvariable=heure)

label.pack(padx=10,pady=10)

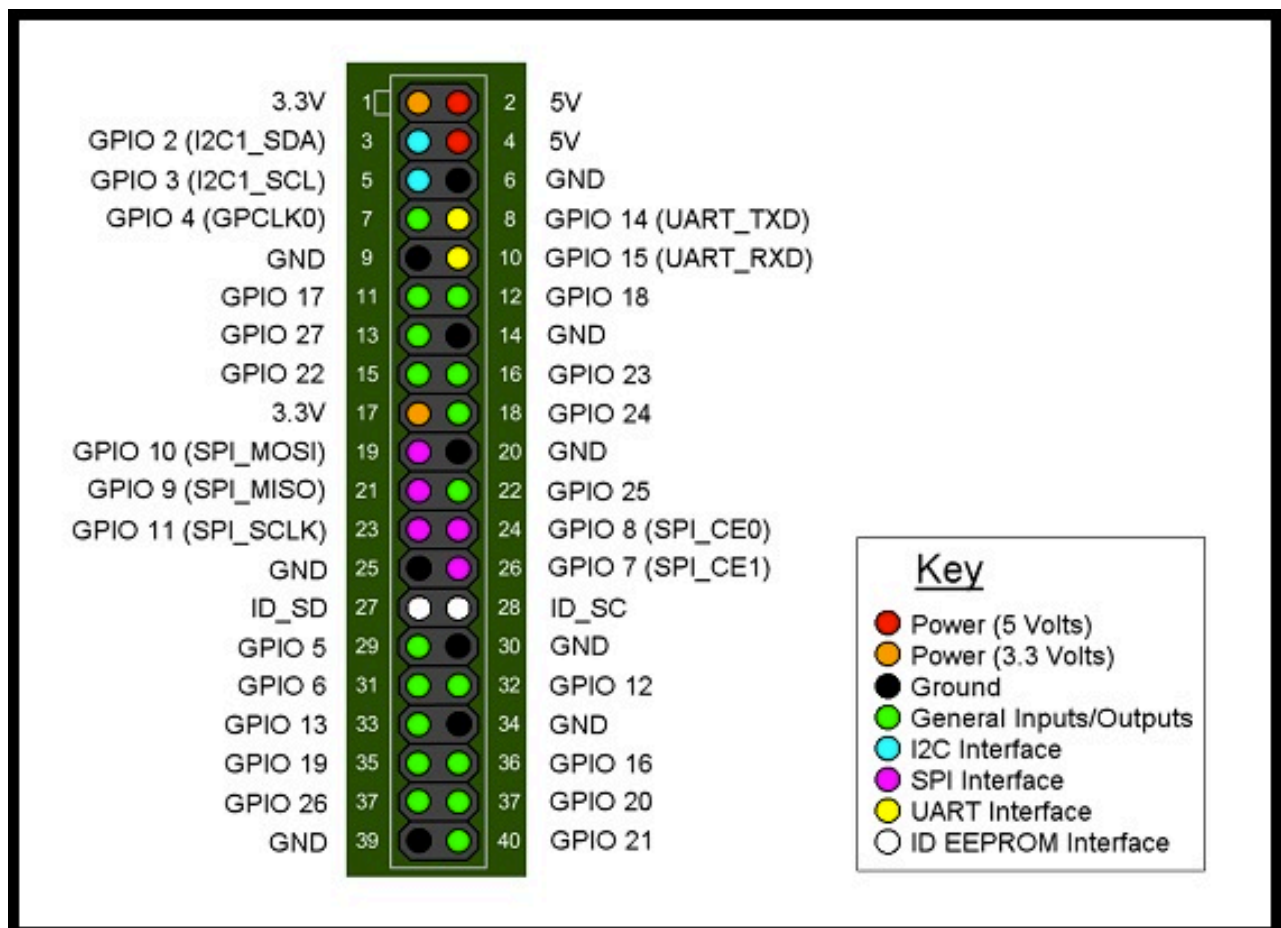
maj()

fenetre.mainloop()

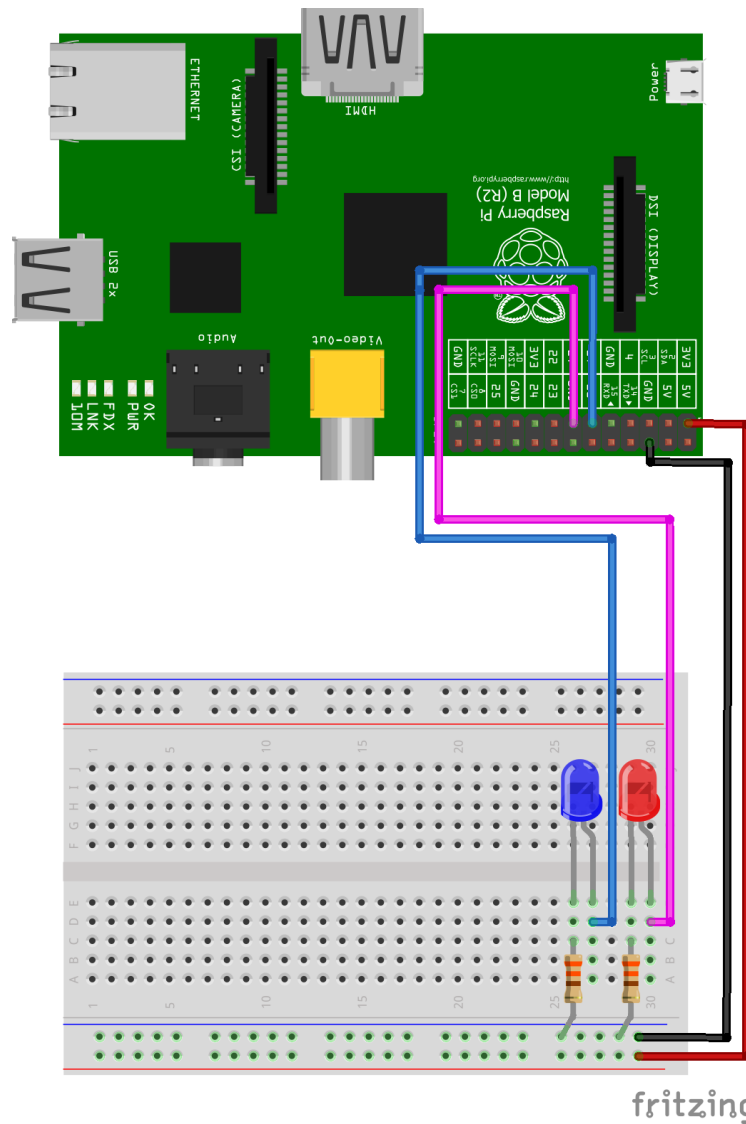
```

V. Interaction avec le monde physique extérieur avec GPIO

Ces activités sont inspirées par : <https://www.modmypi.com/raspberry-pi/set-up-kits/project-kits/raspberry-pi-youtube-workshop-kit>



1. Allumage de 2 diodes



L'alimentation de la Breadboard se fait en reliant Pin 1 (3.3 V) à la ligne + et Pin 6 (Ground) à la ligne -
Brancher les diodes et les résistances comme indiqué sur le schéma sur les Pin 17 et 27.
Recopiez le code ci-dessous dans le TextEditor :

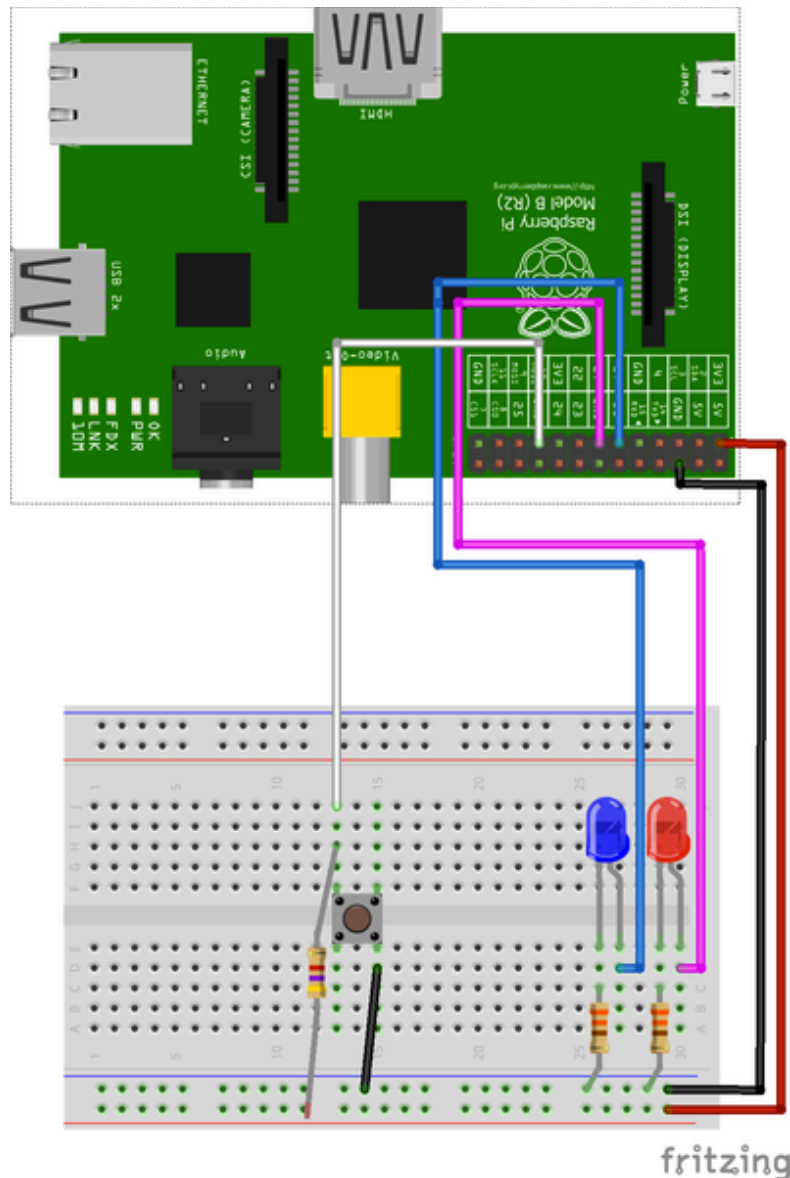
```
#!/usr/bin/python
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)
GPIO.cleanup()
GPIO.setwarnings(False)
GPIO.setup(17,GPIO.OUT)
GPIO.setup(27,GPIO.OUT)
print "Lights on"
GPIO.output(17,GPIO.HIGH)
GPIO.output(27,GPIO.HIGH)
```

Créer un dossier sur le bureau appelé Code_Python dans lequel vous placerez le fichier DiodesOn.py.
Placer dans le même dossier un fichier DiodesOff.py où vous mettrez cette fois les niveaux bas : GPIO.LOW.
Dans le terminal, écrire après `pi@raspberrypi ~ $ : cd Desktop/Code_Python` pour se placer dans ce dossier puis écrire `sudo python DiodesOn.py` puis `sudo python DiodesOff.py`

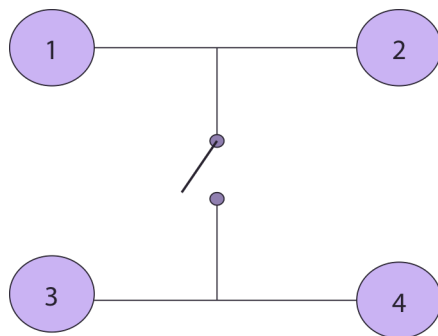
Travail 1/4 : Ecrire un programme qui permet d'éclairer la diode rouge puis la diode bleue 5 fois de suite alternativement toutes les secondes, en utilisant la librairie time et la fonction `time.sleep(1)`.

Travail 2/4 : Ecrire un programme qui demande à l'utilisateur quelle diode il veut allumer et le nombre de fois qu'il veut la voir clignoter. On utilisera deux variables : `led_choice` et `count`, et l'interaction se fera de la manière suivante : `led_choice = input («Choix de la diode : » )`.

2. Un bouton poussoir



Le bouton poussoir se comporte comme un interrupteur qui permet de relier les points 1 et 2 aux points 3 et 4.



Si le bouton n'est pas pressé, le point supérieur gauche 1 est à un potentiel électrique positif inférieur à 3,3 V grâce à la résistance, par contre, si le bouton est pressé, le point 1 est connecté au point 4 et donc son potentiel électrique est nul.

Le GPIO 10 est à l'écoute du potentiel du point 1.

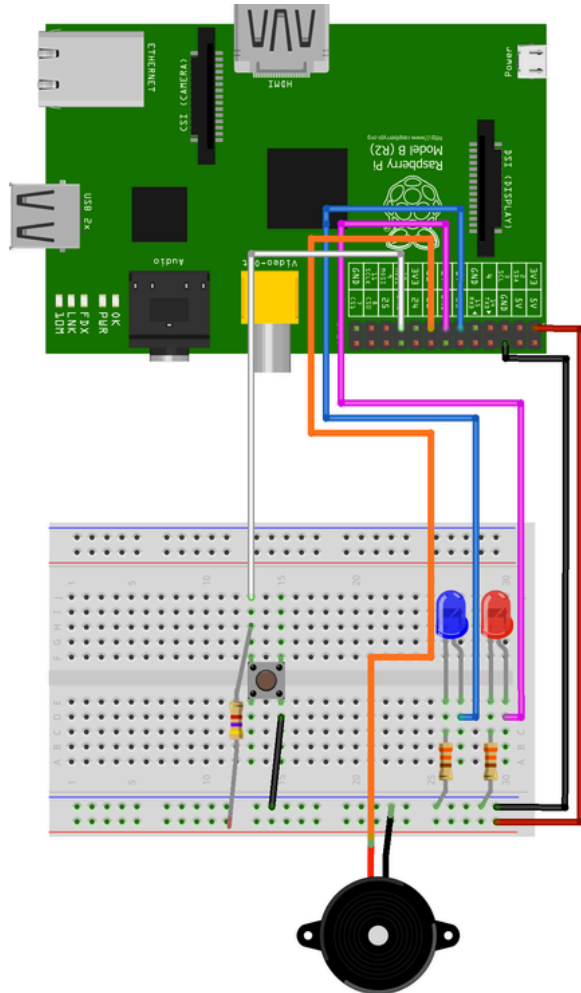
Cette fois, le GPIO est utilisé non plus en sortie OUT comme dans le cas des diodes, mais en entrée IN.

→ Ajouter le fichier Button.py dans le dossier précédent. Réaliser le montage et interpréter le programme.

```
#!/usr/bin/python
import os
import time
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(10, GPIO.IN)
print("-----")
print(" Button + GPIO ")
print("-----")
print GPIO.input(10)
while True:
    if ( GPIO.input(10) == False ):
        print("Button Pressed")
        os.system('date')
        print GPIO.input(10)
        time.sleep(5)
    else:
        os.system('clear')
        print ("Waiting for you to press a button")
        time.sleep(1)
```

N.B : pour arrêter une boucle: ctrl C

3. Un buzzer

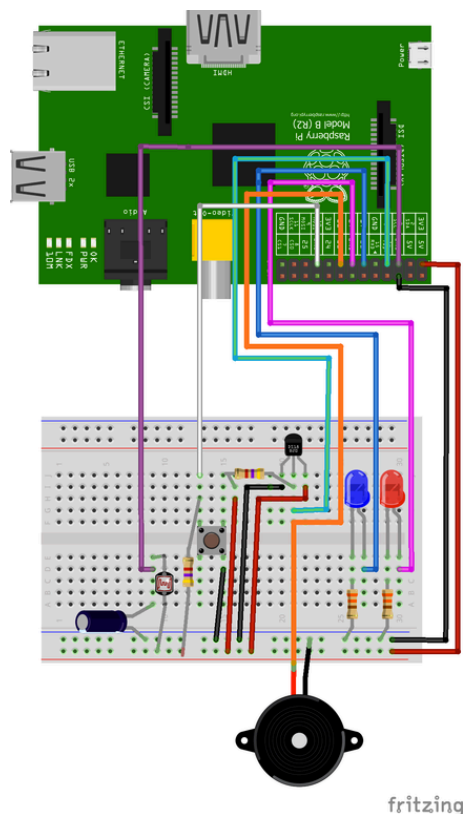


```
#!/usr/bin/python
import os
import time
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(22,GPIO.OUT)
loop_count = 0
def morsecode ():
    GPIO.output(22,GPIO.HIGH)
    time.sleep(.1)
    GPIO.output(22,GPIO.LOW)
    time.sleep(.1)
loop_count = input("How many times would you like to loop?: ")
while loop_count > 0:
    loop_count = loop_count - 1
    morsecode ()
```

→ Ajouter le fichier Buzzer.py dans le dossier précédent. Réaliser le montage et interpréter le programme.

Travail 3/4 : Ecrire un programme qui déclenche le buzzer lorsqu'on appuie sur le bouton. Penser à remettre le GPIO au niveau bas avant de sortir de la boucle avec un break pour éviter une sonnerie intempestive.

4. Une résistance qui dépend de l'éclairement : L.D.R (light-dependent resistor)



```
#!/usr/bin/env python
import os
import datetime
import time
import RPi.GPIO as GPIO
GPIO.setwarnings(False)
DEBUG = 1
```

```

GPIO.setmode(GPIO.BCM)
def RCtime (RCpin):
    reading = 0
    GPIO.setup(RCpin, GPIO.OUT)
    GPIO.output(RCpin, GPIO.LOW)
    time.sleep(1)
    GPIO.setup(RCpin, GPIO.IN)
    # This takes about 1 millisecond per loop cycle
    while (GPIO.input(RCpin) == GPIO.LOW):
        reading += 1
    return reading
while True:
    GetDateTime = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    LDRReading = RCtime(3)
    print RCtime(3)
    # Open a file
    fo = open("/home/pi/10x10/foo.txt", "wb")
    fo.write (GetDateTime)
    LDRReading = str(LDRReading)
    fo.write ("\n")
    fo.write (LDRReading)
    # Close opened file
    fo.close()
    time.sleep(1)

```

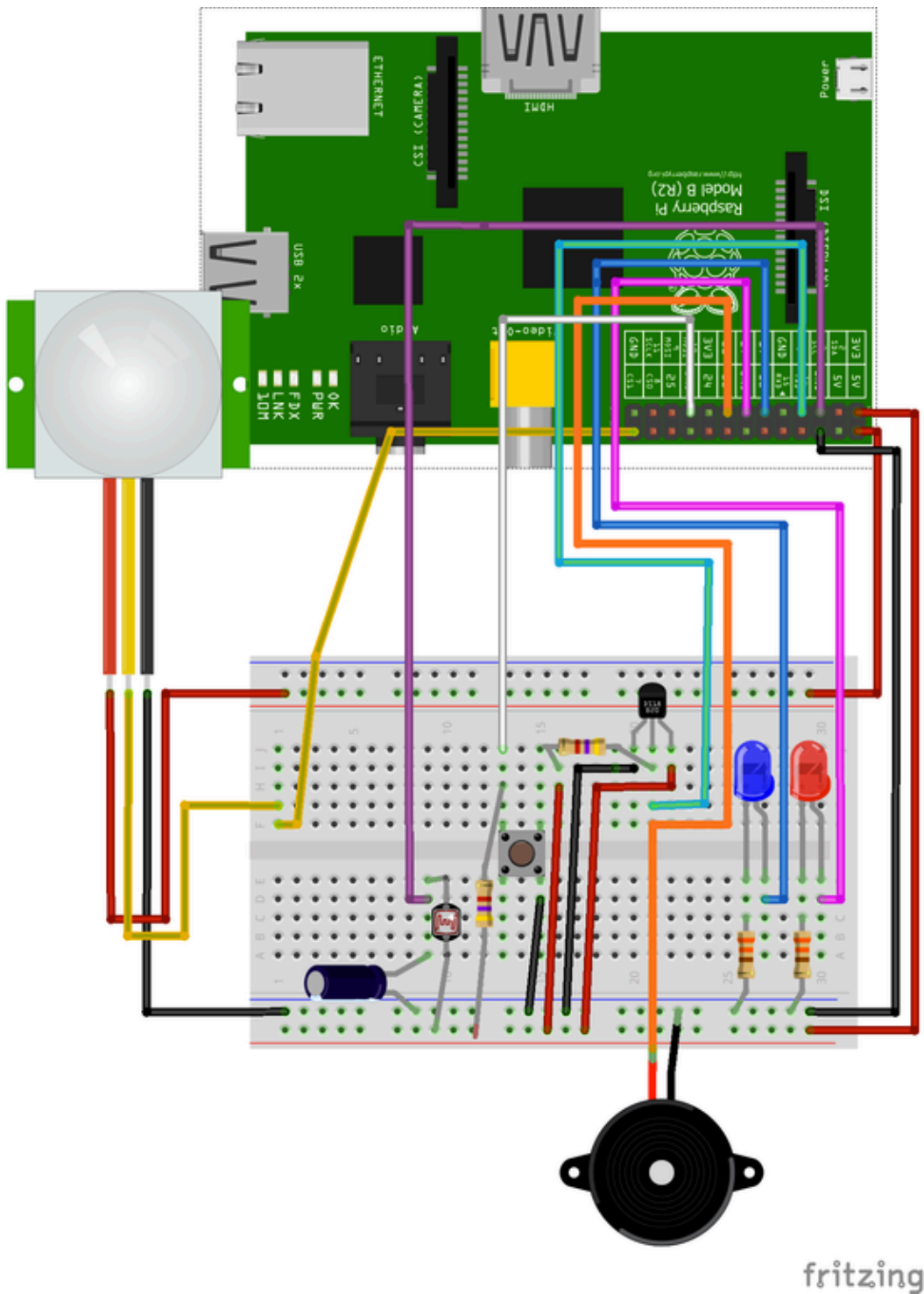
→ Ajouter le fichier LDR.py dans le dossier précédent.

Avant de lancer le programme, créer un fichier texte foo en écrivant touch foo.txt

Le principe de ce montage et de ce programme est de mesurer la durée (au travers d'une variable reading incrémentée de une unité à chaque boucle) que met la tension aux bornes de la LDR en série avec le condensateur pour atteindre le niveau bas du GPIO. Cette mesure est réalisée en boucle. On montre en effet en électronique que cette durée est proportionnelle à la valeur de la résistance, laquelle dépend de l'éclairement reçu. La valeur de reading est affichée dans le fichier foo.txt ainsi que la date courante.

Travail 4/4 : Ecrire un programme qui permet de faire retentir un buzzer si on allume la lumière dans une pièce initialement dans l'obscurité.

5. Un détecteur d'infrarouge



```
#!/usr/bin/python
import RPi.GPIO as GPIO
import time
GPIO.setmode(GPIO.BCM)
GPIO.setwarnings(False)
GPIO.setup(27,GPIO.OUT)
GPIO_PIR = 7
print "PIR Module Test (CTRL-C to exit)"
# Set pin as input
GPIO.setup(GPIO_PIR,GPIO.IN) # Echo
Current_State = 0
Previous_State = 0
```

```

try:
    print "Waiting for PIR to settle ..."
    # Loop until PIR output is 0
    while GPIO.input(GPIO_PIR)==1:
        Current_State = 0
    print " Ready"
    # Loop until users quits with CTRL-C
    while True :
        # Read PIR state
        Current_State = GPIO.input(GPIO_PIR)
        if Current_State==1 and Previous_State==0:
            # PIR is triggered
            print " Motion detected!"
            # Record previous state
            GPIO.output(27,GPIO.HIGH)
            time.sleep(1)
            GPIO.output(27,GPIO.LOW)
            Previous_State=1
        elif Current_State==0 and Previous_State==1:
            # PIR has returned to ready state
            print " Ready"
            Previous_State=0
            # Wait for 10 milliseconds
            time.sleep(0.01)
except KeyboardInterrupt:
    print " Quit"
    # Reset GPIO settings
    GPIO.cleanup()

```

6. Un capteur de température

Une mise à jour étant nécessaire, faire `sudo /nano/boot/config.txt`

Ajouter : `dtoverlay=w1-gpio`

Suivre le protocole bien expliqué ici :

<http://rasberry-pi.developpez.com/cours-tutoriels/capteur/mag-pi-utiliser-port-gpio/partie-2-detection-temperature-1-wire/>